

Answerspot — Technical Design Specification

AI answer-monitoring platform for local service businesses. Tells operators (roofers, dentists, HVAC, auto shops) whether AI assistants recommend them, who's beating them, and exactly what to fix.

Status: Design → Build handoff **Owner:** Product / Eng Lead **Target infra:** Cloud-agnostic (Docker + Kubernetes) **Last updated:** 2026-07-10

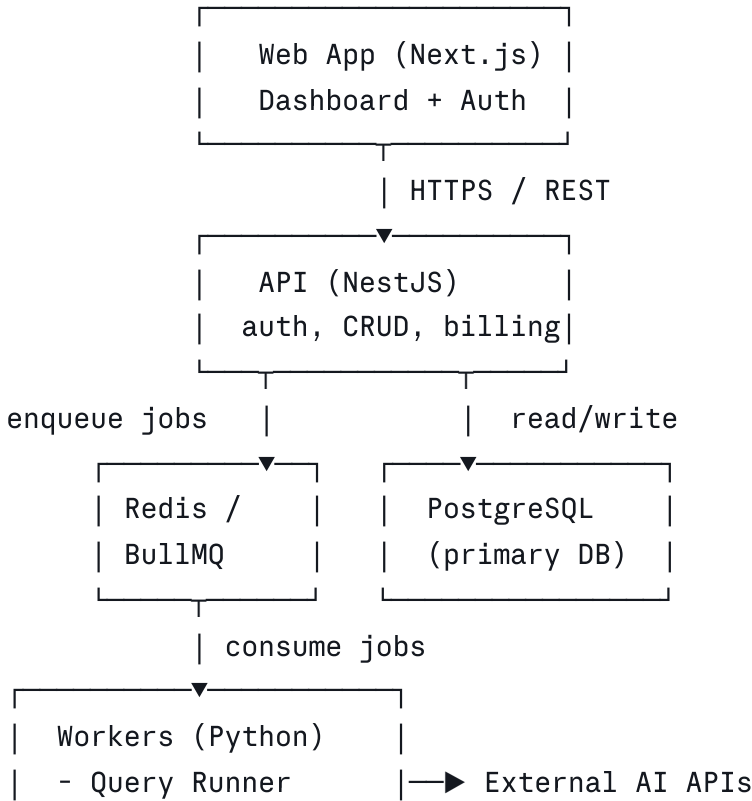
1. Product Summary

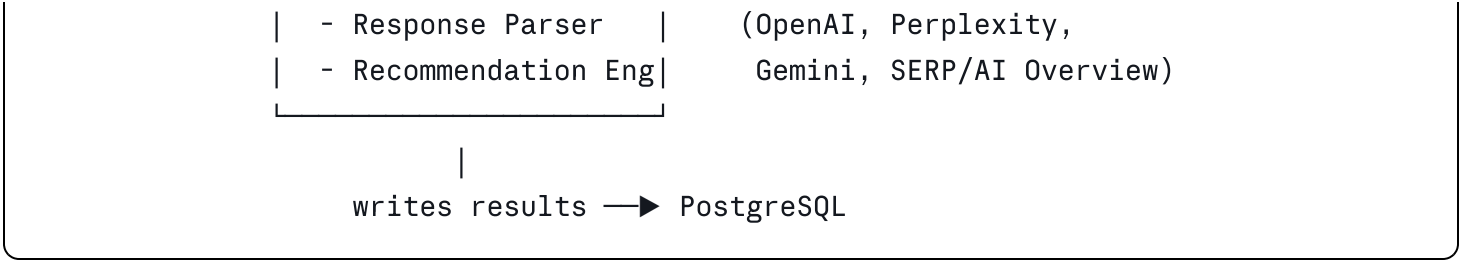
AI search replaced the 10 blue links. When a homeowner asks a chat assistant "best roofer near me," it returns 3 names. Answerspot monitors what AI platforms (ChatGPT, Perplexity, Gemini, Google AI Overviews) say for the queries a business cares about, scores its visibility, ranks it against competitors, and generates plain-language fixes.

Core loop (MVP): Onboard → auto-generate queries → scan platforms → show real answers + Visibility Score → deliver 3 actionable recommendations.

Non-goals for v1: ad buying, generic SEO auditing, social media management, review generation.

2. Architecture Overview





Stack decisions

Layer	Choice	Rationale
Frontend	Next.js 14 (App Router), TypeScript, Tailwind, shadcn/ui	SSR dashboard, fast, strong ecosystem
API	NestJS (Node/TS)	Structured, DI, decorators, OpenAPI out of the box
Workers	Python (Celery or RQ)	Best tooling for LLM calls, parsing, prompt logic
Queue/Cache	Redis + BullMQ (API-side jobs) / Celery broker (workers)	Reliable scheduling + retries
DB	PostgreSQL 16	Relational + JSONB for raw responses
Object storage	S3-compatible (MinIO local / any cloud)	Raw response archives, PDF reports
Auth	Auth.js (or Clerk) w/ JWT	Multi-tenant orgs + roles
Payments	Stripe	Subscriptions + webhooks
Infra	Docker, Kubernetes, Helm	Cloud-agnostic, portable
CI/CD	GitHub Actions	Build, test, scan, push, deploy
Observability	OpenTelemetry → Prometheus/Grafana + Sentry	Metrics, traces, error tracking

3. Repository Structure (Monorepo)



```
└─ docs/
  │ └─ DESIGN_SPEC.md           # this document
  │ └─ ARCHITECTURE.md
  │ └─ API.md                   # generated from OpenAPI
  │ └─ RUNBOOK.md              # on-call / ops
  │ └─ ADR/                     # architecture decision records
└─ apps/
  │ └─ web/                     # Next.js frontend
  │ └─ api/                     # NestJS API
  │ └─ workers/                 # Python workers
└─ packages/
  │ └─ shared-types/           # shared TS types (API <-> web)
  │ └─ ui/                     # shared React components
  │ └─ config/                  # eslint, tsconfig, prettier
└─ db/
  │ └─ migrations/             # Prisma or SQL migrations
  │ └─ seed/
└─ infra/
  │ └─ docker/                 # Dockerfiles per service
  │ └─ k8s/                    # base manifests
  │ └─ helm/                   # Helm charts
  │ └─ terraform/              # optional cloud provisioning
└─ .github/
  │ └─ workflows/              # CI/CD pipelines
└─ docker-compose.yml          # local dev (all services)
└─ .env.example
└─ Makefile                     # dev shortcuts
```

4. Data Model

PostgreSQL. Multi-tenant via `organization_id` on all tenant-scoped tables. Timestamps (`created_at`, `updated_at`) on every table; soft-delete (`deleted_at`) where noted.

sql

```

-- Tenancy & users
organizations (id PK, name, plan, stripe_customer_id, status, created_at)
users (id PK, organization_id FK, email UNIQUE, name, role, auth_provider_id FK, created_at)
memberships (id PK, user_id FK, organization_id FK, role) -- owner|admin|member

-- Domain
businesses (id PK, organization_id FK, name, category, city, state, website, gbp_id, created_at, deleted_at)
platforms (id PK, key, display_name, enabled) -- chatgpt|perplexity
tracked_queries (id PK, business_id FK, query_text, location, intent, is_active, created_at)

-- Scanning
scans (id PK, tracked_query_id FK, platform_id FK, run_at, status, samples, error, created_at) -- status: queued|running
scan_results (id PK, scan_id FK, raw_response TEXT, mentioned BOOL, rank_position INT, sentiment TEXT, confidence NUMERIC, citations JSONB, competitors JSONB, created_at)

-- Insights
visibility_scores (id PK, business_id FK, score INT, breakdown JSONB, computed_at)
recommendations (id PK, business_id FK, type, severity, title, message, artifact JSONB, status, created_at) -- status: open|resolved
alerts (id PK, business_id FK, type, payload JSONB, sent_at, channel)

-- Billing
subscriptions (id PK, organization_id FK, stripe_subscription_id, plan, status, current_period_end)

```

Use **Prisma** for the API's typed access + migrations. Workers read/write via SQLAlchemy against the same schema (Prisma owns migrations; workers do not migrate).

5. Core Services

5.1 Query Runner (Python worker)

- Consumes `scan.run` jobs. For each `tracked_query × platform`, sends the query N times (default 3 samples) to smooth non-deterministic answers.
- Adapter pattern per platform (`OpenAIAdapter`, `PerplexityAdapter`, `GeminiAdapter`, `AIOverviewAdapter`). Each returns a normalized `RawAnswer`.

- Rate-limited, retried with backoff, results stored raw before parsing.

5.2 Response Parser (Python worker)

- Consumes `scan.parse` jobs. Uses an LLM with a strict JSON schema to extract: `mentioned`, `rank_position`, `sentiment`, `citations[]`, `competitors[]`.
- Validates output against a Pydantic schema; retries on invalid JSON.
- Aggregates the N samples into one `scan_result` with a `confidence` value.

5.3 Recommendation Engine (Python worker)

- Runs after parsing. Compares the business's presence vs competitors, checks citation gaps, review/sentiment signals, and schema-markup gaps.
- Emits ranked `recommendations`, generating an **artifact** where possible (e.g., FAQ schema snippet, homepage keyword list, review-response draft).

5.4 Scheduler

- Cron-driven (K8s CronJob or Celery beat) enqueues scans per plan tier: Starter = weekly, Pro/Agency = daily.

5.5 Visibility Score

Deterministic function of: appearance rate, avg rank position, sentiment, citation strength. Stored as a snapshot per compute so trend lines are historical and immutable.

```
score = w1·appearanceRate + w2·rankScore + w3·sentimentScore + w4·citationScore  
(weights configurable; documented in ADR)
```

6. API Surface (v1)

Base: `/api/v1`. Auth: Bearer JWT. All list endpoints paginated + org-scoped.

POST	/auth/session	login / token exchange
GET	/me	
POST	/businesses	create + trigger onboarding scan
GET	/businesses	
GET	/businesses/:id	
GET	/businesses/:id/visibility-score	current + trend
GET	/businesses/:id/competitors	
GET	/businesses/:id/recommendations	
POST	/businesses/:id/queries	add tracked query
GET	/businesses/:id/queries	
PATCH	/queries/:id	activate/deactivate/edit
GET	/queries/:id/results?platform=&range=	
POST	/scans/trigger	manual scan (rate-limited)
GET	/scans/:id	
GET	/reports/:businessId?format=pdf	white-label report
POST	/webhooks/stripe	billing events (raw body, signature-verified)

OpenAPI spec auto-generated by NestJS → published to [docs/API.md](#).

7. Frontend (Dashboard)

- **Home:** Visibility Score (hero), trend line, top 3 recommendations, competitor summary, recent answer changes.
- **Answer Explorer:** real AI responses side-by-side per platform; business mentions and competitors highlighted.
- **Competitors:** you vs top 3–5 rivals across every query.
- **Recommendations:** ranked, plain-language, each with a copyable artifact and resolve/dismiss.
- **Queries:** manage tracked queries.
- **Reports:** generate/branded export (Agency tier).
- **Settings/Billing:** Stripe portal, team, plan.

Design principles: plain language over jargon, speed-to-first-insight in onboarding, confidence ranges not false precision.

8. Non-Functional Requirements

- **Multi-tenancy:** every query filtered by `organization_id`; row-level checks in a guard/interceptor.
 - **Security:** JWT auth, RBAC (owner/admin/member), secrets via K8s Secrets/Vault, Stripe webhook signature verification, no PII in logs.
 - **Reliability:** worker jobs idempotent + retried with backoff; dead-letter queue for failures.
 - **Cost control:** cache identical query results within a scan window; sample count configurable per plan; budget caps on external API spend.
 - **Performance:** dashboard reads served from precomputed snapshots, not live LLM calls.
 - **Compliance/honesty:** all scores labeled directional; disclaimer surfaced in UI and reports.
-

9. Infrastructure & Deployment

Local dev

`docker-compose up` brings up: web, api, workers, postgres, redis, minio. `Makefile` targets: `make dev`, `make migrate`, `make seed`, `make test`.

Kubernetes (prod)

- Namespaces: `answerspot-prod`, `answerspot-staging`.
- Deployments: `web`, `api`, `worker-runner`, `worker-parser`, `worker-recs`.
- CronJob: `scheduler`.
- StatefulSet/managed: `postgres`, `redis` (recommend managed DB in cloud; self-hosted acceptable via Helm for portability).
- Ingress + TLS (cert-manager).
- HPA on api + workers (CPU/queue depth).
- Config via ConfigMaps; secrets via Sealed Secrets or external Vault.
- Helm chart parameterizes image tags, replicas, env per environment.

CI/CD (GitHub Actions)

```
on PR:  lint → typecheck → unit tests → build → security scan (trivy) → docker
build (no push)
on main: above + push images to registry → deploy staging (Helm) → smoke tests
on tag:  promote to prod (manual approval gate) → Helm upgrade → post-deploy
healthcheck
```

10. Environment Variables (`.env.example`)

```
# API
DATABASE_URL=postgres://...
REDIS_URL=redis://...
JWT_SECRET=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
S3_ENDPOINT=  S3_BUCKET=  S3_KEY=  S3_SECRET=

# Workers
OPENAI_API_KEY=
PERPLEXITY_API_KEY=
GEMINI_API_KEY=
SERP_API_KEY=          # AI Overview capture
SCAN_SAMPLE_COUNT=3

# Web
NEXT_PUBLIC_API_URL=
```

11. Build Milestones

Phase	Deliverable
M0	Repo scaffold, docker-compose, CI skeleton, DB schema + migrations
M1	Auth + org/business onboarding + auto-query generation
M2	Query Runner (Perplexity first) + scan storage
M3	Response Parser + Visibility Score
M4	Dashboard: Home, Answer Explorer, trend
M5	Competitor Radar + Recommendation Engine + artifacts
M6	Multi-platform, alerts, Stripe billing
M7	White-label reports (Agency), HPA/observability hardening

Fundable MVP = M0-M4 (the "oh wow" loop). M5+ is monetization and expansion.

12. Open Decisions (record as ADRs)

1. Auth: Auth.js (self-hosted, portable) vs Clerk (faster, vendor lock).
 2. Managed vs self-hosted Postgres/Redis in prod.
 3. Python worker framework: Celery vs RQ.
 4. AI Overview capture method (SERP API vs headless browser) — ToS/legal review required.
 5. Visibility Score weightings — needs data to calibrate.
-

13. Handoff Checklist for DevOps/Eng

- ☐ Provision registry + K8s cluster (or confirm target)
- ☐ Set up secrets management (Vault/Sealed Secrets)
- ☐ Configure GitHub Actions runners + environments (staging/prod gates)
- ☐ Stand up managed Postgres + Redis or Helm equivalents
- ☐ Wire external API keys with budget alerts
- ☐ Baseline observability (Prometheus/Grafana/Sentry)
- ☐ Review ToS/legal for each AI platform scraping/API use